The Switch Is the Bootstrap: Deep Private Inference at Zero FHE Bootstraps via Scheme Switching

(author names elided for review)

Abstract

Hybrid private-inference systems evaluate a neural network's linear layers under homomorphic encryption (FHE) and its nonlinearities under secure multi-party computation (MPC), switching representation between the two. Prior work treats this scheme switch as an engineering necessity. We observe that it is also a *noise-reset mechanism*: the re-encryption that returns an activation from MPC shares to an FHE ciphertext produces a ciphertext distributed identically to a *fresh*, depth-one encryption. Consequently a network of *arbitrary depth* is evaluated with **zero FHE bootstraps** and cost **linear in depth**, using an FHE engine that supports only a *single* homomorphic multiplication and *no evaluation keys*. We make the argument precise (Theorem 1), instantiate the FHE side so that a full matrix–vector product with *both* the weights and the activations encrypted is one ciphertext×ciphertext multiplication decrypted under s^2 (no relinearization), and give a convolution packing that yields an entire output channel per multiplication. The MPC side is a triple-free, communication-batched honest-majority protocol. We implement the whole pipeline and validate it by *bit-exact* agreement with a plaintext integer reference at every depth. On real MNIST test digits, a fully encrypted two-convolution CNN — four encrypted linear layers, three MPC nonlinearities, three scheme switches, and zero bootstraps — attains 98.05% test accuracy, with every encrypted prediction provably equal to its plaintext counterpart. Unlike two-party inference services, the model weights are encrypted as well as the input.

1 Introduction

Fully homomorphic encryption [15] lets a server compute on encrypted data, but every homomorphic multiplication grows ciphertext noise; after a bounded number of multiplications the ciphertext must be *bootstrapped* — an expensive homomorphic decryption — to continue. Deep networks therefore incur bootstrapping cost that scales with depth.

An orthogonal line of work evaluates the *linear* layers under FHE and the *nonlinear* layers (ReLU, max-pooling, argmax) under MPC [18, 24, 16, 26], because comparisons are cheap in MPC and expensive in FHE. Between the two representations sits a *scheme switch*: mask the ciphertext, decrypt the mask, compute the nonlinearity on additive secret shares, and re-encrypt the result. These systems present the switch as unavoidable overhead.

Our observation. The return leg of the switch — re-encrypting each party's share component and summing — produces a ciphertext that is a sum of *fresh* encryptions. A sum of fresh encryptions is itself a fresh encryption: it carries only fresh noise and sits at multiplicative depth one, indistinguishable from a first-layer input. The switch that a system already performs for the ReLU therefore *also refreshes the FHE ciphertext*, and it does so for free. The per-layer FHE noise

budget is a constant *independent of network depth*, so bootstrapping is never needed — not deferred, but absent. We call this "the switch is the bootstrap."

This inverts a familiar tradeoff. Because the FHE engine only ever performs one multiplication before the next switch resets it, we may use an FHE scheme that is fast precisely *because* it supports only depth one and requires *no evaluation keys* (no relinearization keys, no Galois keys, no bootstrapping keys). A ciphertext×ciphertext product is a degree-2 ciphertext; rather than relinearizing it back to degree one, we simply decrypt it under s^2 at the next switch — which opens a masked value anyway. The result is a private-inference construction whose FHE cost per layer is a single multiplication and whose total FHE cost is linear in depth with a small constant.

Contributions.

1. **Switch-as-bootstrap (Section 3).** We formalize the claim that the MPC round trip refreshes the FHE ciphertext to a depth-one state, giving a per-layer noise bound independent of depth and hence unbounded-depth evaluation at zero bootstraps. The argument is scheme-agnostic: any additively-homomorphic FHE with a cheap depth-one multiply instantiates it.

2. **Evaluation-key-free, both-operands-private linear layers (Section 4).** Instantiating the FHE side with RLWE (BFV) and reversed-coefficient packing, a full matrix–vector product with both the weight matrix *and* the activation vector encrypted is a single ciphertext×ciphertext multiplication decrypted under s^2, with no relinearization and no evaluation keys. A Cheetah-style convolution packing yields an entire output channel per multiplication. This strengthens the privacy model of prior hybrid systems, which keep the model in plaintext.

3. **A triple-free, batched honest-majority MPC layer (Section 5).** Multiplication uses ABY3-style resharing with PRF-derived correlated randomness — one ring element per party per multiplication, no Beaver triples, no dealer. Bit decomposition is a Kogge–Stone construction batched across an entire layer, so a ReLU or max-pooling layer costs a constant number of communication rounds regardless of width.

4. **Composability (Section 6).** The pipeline is an Arithmetic Black Box; a small gadget library (`relu`, `max2`, `maxpool2d`, `argmax`, `dense`, `conv2d`) realizes standard network layers as pure functions of the ABB primitives, so any architecture built from them inherits security and correctness with no per-architecture proof.

5. **An implementation validated bit-exactly (Section 8).** We implement the entire construction and check it against a plaintext integer reference: the encrypted output equals the plaintext output *bit for bit* at every measured depth. On real MNIST test digits, a fully encrypted two-convolution CNN reaches 98.05% accuracy at zero bootstraps.

Honesty about scope. We separate what is proven, what is implemented, and what is simulated. The design is semi-honest, honest-majority. Two aspects of the *implementation* stand in for their deployable form and are disclosed as limitations (Section 7): the masked decryption uses a single client key as a placeholder for threshold FHE, and the fast reference FHE engine (scale-invariant NTRU) is *insecure* at the parameters this construction needs. The secure instantiation is RLWE, on which all headline accuracy and correctness numbers are measured; NTRU is retained only as a speed reference for isolating packing cost from FHE-scheme cost.

2 Preliminaries

Notation. We work over the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ with n a power of two. Lowercase letters denote ring elements; s is an RLWE secret key. For an integer v and width b, $\langle v \rangle$ denotes an additive secret sharing over $\mathbb{Z}_{2^b}$. We write $x \gg s$ for a logical right shift by s bits.

RLWE encryption (BFV). We use the Brakerski/Fan–Vercauteren scheme [14]. A plaintext is a polynomial with coefficients mod t; a ciphertext is a pair $(c_0, c_1) \in R_q^2$ with $c_0 + c_1 s \approx \Delta m$ for $\Delta = \lfloor q/t \rfloor$. Addition is componentwise. The product of two ciphertexts is a *degree-2* ciphertext (d_0, d_1, d_2) satisfying $d_0 + d_1 s + d_2 s^2 \approx \Delta^2 m_1 m_2 / \Delta$ after rescaling; usually one *relinearizes* (d_0, d_1, d_2) back to a pair using an evaluation key. We never relinearize: the next scheme switch decrypts the degree-2 ciphertext directly under s^2 (Section 4). We take $n = 4096$ and $\log_2 q = 100$, at or below the 128-bit security ceiling of $\log_2 q \leq 109$ for $n = 4096$ [1].

Replicated secret sharing (ABY3). For the honest-majority three-party segment we use 2-out-of-3 replicated sharing [25]: a value $v = v_0 + v_1 + v_2$ over $\mathbb{Z}_{2^b}$ is shared so party i holds (v_i, v_{i+1}). Addition and multiplication-by-constant are local. Multiplication is described in Section 5.

Scheme switching. A *switch to MPC*, $\mathsf{fhe}\to\mathsf{shares}$, takes an FHE ciphertext of a value x, has the parties add fresh random masks, decrypts the single masked ciphertext, and produces an additive sharing $\langle x \rangle$; only a uniformly-masked value is ever opened. A *switch to FHE*, $\mathsf{shares}\to\mathsf{fhe}$, has each party freshly encrypt its share component; the evaluator sums the ciphertexts.

3 The Switch-as-Bootstrap Construction

3.1 The layer template

A network is a sequence of layers $x \mapsto \sigma(L(x))$, where L is an affine map (dense or convolution, possibly with encrypted weights) and σ is a nonlinearity (ReLU, optionally followed by max-pooling). Each layer is evaluated as:

1. **FHE linear map.** Given a fresh ciphertext $c = \mathsf{Enc}(x)$, compute $c' = L(c)$: one homomorphic multiplication (Section 4). c' is a degree-2 ciphertext with depth-one noise.
2. **Switch to MPC.** $\langle y \rangle \leftarrow \mathsf{fhe}\to\mathsf{shares}(c')$, opening only a masked value. The ciphertext noise is now discarded.
3. **Nonlinearity in MPC.** $\langle a \rangle \leftarrow \sigma(\langle y \rangle)$ (Section 5).
4. **Switch to FHE.** $c_{\text{next}} \leftarrow \mathsf{shares}\to\mathsf{fhe}(\langle a \rangle)$: a fresh ciphertext for the next layer.

3.2 Main claim

Theorem 1 (Switch-as-bootstrap)**.** *Let* Enc *be an additively homomorphic encryption scheme whose fresh ciphertexts carry noise at most* ν_0 *and admit one homomorphic multiplication within a decryption-correct noise budget* B *(i.e.* $\text{poly}(\nu_0) \leq B$*). In the layer template above, the ciphertext entering the FHE step of* every *layer is distributed identically to a fresh encryption: it has multiplicative depth one and noise at most* $w \cdot \nu_0$*, where* w *is the layer's fan-in width. Consequently, for a network of any depth* D*, decryption correctness holds at every layer under the single budget* B*, and no bootstrapping is invoked.*

Proof sketch. The claim is an invariant maintained by induction on the layer index. The base case is the network input, a fresh encryption by construction. For the inductive step, assume layer ℓ begins with a fresh ciphertext. Steps 1–3 consume it and produce an additive sharing $\langle a \rangle$ living entirely in the MPC domain; the FHE ciphertext and its noise are discarded at Step 2. Step 4 forms $c_{\text{next}} = \sum_i \mathsf{Enc}(a^{(i)})$, a sum of freshly sampled encryptions of the parties' share components $a^{(i)}$. By additive homomorphism c_{next} encrypts $\sum_i a^{(i)} = a$, and because each summand is an independent fresh encryption, c_{next} is distributed as a fresh encryption of a with noise bounded by the sum of at most w fresh noises, i.e. $w \cdot \nu_0$, at multiplicative depth one. Thus layer $\ell + 1$ begins in the same fresh state. Since the state is fresh at every layer, the single-multiplication budget B suffices at every layer, independent of D. Bootstrapping, the operation used only to reset accumulated noise, is therefore never invoked. □

Remark 1 (Cost linear in depth). Each layer performs a constant amount of FHE work (one multiply, $O(1)$ mask ciphertexts), one masked decryption, and one MPC nonlinearity. Total cost is therefore linear in depth, with no super-linear bootstrapping term. This is the qualitative separation from leveled FHE, where the per-multiply cost grows with the remaining level budget or a bootstrap is amortized in.

Remark 2 (Why we can drop evaluation keys). Theorem 1 needs only *one* multiplication per layer and never composes two multiplications homomorphically. Relinearization exists to make a degree-2 product usable by a *subsequent* homomorphic operation; here the subsequent operation is a decryption (the switch), which handles a degree-2 ciphertext natively by evaluating $d_0 + d_1 s + d_2 s^2$. No relinearization key is needed. Coefficient packing (Section 4) avoids slot rotations, so no Galois keys are needed either. And no bootstrapping key is needed by the theorem. The FHE public material is a single public key.

3.3 Correctness of the fixed-point values

Theorem 1 concerns ciphertext *noise*. Independently, the integer activations must not overflow the share modulus 2^b. We use a fused nonlinearity

$$a' \;=\; \mathrm{ReLU}\big((W \cdot a + (\beta \ll s)) \gg s\big), \tag{1}$$

where the arithmetic right shift by s rescales the fixed-point product so that activation magnitude stays roughly constant across layers. Two points make this exact.

First, the rescale is a *logical* shift, yet $W \cdot a + (\beta \ll s)$ can be negative before the ReLU. Correctness holds because the shifted value is retained only on the non-negative branch of the ReLU — where a logical and an arithmetic shift agree — and zeroed on the negative branch. Hence a single bit decomposition of the pre-activation yields *both* the sign bit (for the ReLU) and the shifted magnitude (for the rescale): one bit decomposition per layer, not two.

Second, the construction is *exact* integer fixed-point. The encrypted output equals the plaintext integer network bit for bit — not up to an approximation error — which is what our experiments verify as `correct = true` (Section 8). This is a much stronger correctness statement than the approximate equality reported by CKKS-based approximate inference.

4 Evaluation-Key-Free Linear Layers

4.1 Both operands encrypted, no relinearization

We pack a length-m vector into the coefficients of a degree-m polynomial. To compute an inner product $\langle u, v \rangle$, we pack u in natural order and v in *reversed* order; the degree-$(m-1)$ coefficient

of the polynomial product $u(X) \cdot v(X)$ is exactly $\sum_i u_i v_i$. Both polynomials are ciphertexts, so a matrix–vector product is a ciphertext×ciphertext multiplication in which *both* the weights and the activations are encrypted. The product is a degree-2 ciphertext, which the next switch decrypts under s^2; we never relinearize.

For a matrix with ρ output rows and inner dimension m, the reversed layout places row r's weights at coefficient offset $r \cdot m$, so a single degree-n product yields up to $\lfloor n/m \rfloor - 1$ separable outputs; a layer of ρ rows costs $\lceil \rho m/n \rceil$ multiplications. We show this is optimal for the coefficient encoding: a degree-n product admits at most n/m separable outputs because distinct rows require disjoint weight windows. SIMD-slot packing matches the throughput but requires rotations and Galois keys, forfeiting the evaluation-key-free property.

4.2 Exact negacyclic products via a multi-modular NTT

For a power-of-two modulus q the ring R_q has no primitive $2n$-th root of unity, so no direct NTT exists. We instead compute the *exact* integer negacyclic product using several NTT-friendly primes as a computational device and reconstruct the integer coefficients by the Chinese Remainder Theorem (Garner's algorithm), then reduce mod q (and apply the rounded-shift rescale). This is differentially tested against a schoolbook reference at full coefficient width and $n = 4096$. It accelerates the BFV multiplication by roughly 11× over the schoolbook implementation it replaces.

4.3 Convolution packing: an output channel per multiplication

Polynomial multiplication is itself a convolution, which we exploit directly. Flatten a $C \times H \times W$ input row-major into coefficients and pack a kernel *reversed on every axis* (channel, row, column) at the input's strides. Then a single ciphertext×ciphertext product places the valid convolution output for an entire output channel at a fixed set of coefficients. Two facts make it exact:

- **Channels pair off.** The reversed channel offset makes $c + (C-1-c') = C-1$ hold only when $c = c'$, so every input channel's contribution lands on the same output coefficient and they sum — exactly what a multi-channel convolution requires.
- **No row-wrap contamination.** A neighbouring row could only leak into an output coefficient via a column carry, which would require a negative output column index — impossible for a valid output. The read-out coefficients are therefore clean.

The layer costs out_c multiplications, *independent of image size*, versus $\lceil \text{out_c} \cdot o_h o_w \cdot CHW/n \rceil$ for an im2col/Toeplitz formulation. At 28×28 with a 3×3 kernel and two output channels this is 2 multiplications instead of 1352.

5 The MPC Layer

5.1 Triple-free multiplication

With replicated sharing $x = x_0 + x_1 + x_2$, each party i holds (x_i, x_{i+1}) and can form three of the nine cross terms of $x \cdot y$ locally:

$$z_i \;=\; x_i y_i + x_i y_{i+1} + x_{i+1} y_i + \alpha_i, \tag{2}$$

where the α_i are zero-sum correlated randomness derived locally from pairwise PRF keys, $\alpha_i = F(k_i, \text{ctr}) - F(k_{i-1}, \text{ctr})$. Summing over the three parties covers all nine cross terms exactly once,

so $\sum_i z_i = (\sum_i x_i)(\sum_i y_i) = xy$; the α_i cancel in the sum but re-randomize each party's share so nothing leaks. Each multiplication is one ring element sent to one neighbour — no Beaver triples [4], no dealer, nothing precomputed beyond a one-time key handshake. The PRF counter is a globally distinct value per multiplication, so no two products ever share an α (which would expose their difference).

5.2 Batched bit decomposition

Comparisons (for ReLU, max-pooling, argmax) reduce to extracting the sign bit of a shared value, which requires an arithmetic-to-binary decomposition. We use the ABY3 decomposition: (i) binary-share the three additive components for free (each is already known to exactly the two parties a replicated sharing needs); (ii) reduce three shares to two with a carry-save adder, exploiting that in this encoding $x \oplus y \oplus z$ and $\mathrm{maj}(x, y, z)$ fall out of the *same* four products (two rounds, not four); (iii) add the remaining two with a Kogge–Stone parallel-prefix adder [20], whose (g, p) combine $G = g \oplus (p \wedge g')$ is local because a group cannot both generate and propagate a carry. The depth is $2 + 1 + \lceil \log_2 b \rceil + 1 \approx 9$ rounds at $b = 20$.

Crucially, we *batch* the decomposition across an entire layer: every internal multiplication round is flattened over all lanes, so decomposing a whole ReLU or max-pooling layer costs the same ≈ 9 rounds as decomposing a single value. Without batching a 1352-neuron ReLU layer would run $\approx 12{,}000$ sequential rounds; with it, ≈ 9. This single change is a 6.7× end-to-end speedup on the MPC segment (Section 8).

6 Composability

The construction manipulates secret values only through opaque handles (a share reference or a ciphertext handle) and a fixed set of primitive operations; it never inspects the representation. This is exactly the Arithmetic Black Box abstraction [11, 12]: an ideal functionality $\mathcal{F}_{\mathsf{ABB}}$ storing values by handle and exposing $\{\mathsf{add}, \mathsf{mul}, \mathsf{lt}, \mathsf{share_public}, \mathsf{reveal}\}$ plus an internal, invisible representation switch. The scheme switch of Section 3 is that internal operation; it is never observable at the ABB interface.

A small gadget library realizes standard layers as pure functions of the ABB primitives (Table 1). Because each gadget is a composition of ABB operations, if each primitive UC-realizes [8] its $\mathcal{F}_{\mathsf{ABB}}$ operation, each gadget UC-realizes the corresponding ideal gadget, and any network — any sequence of gadgets — UC-realizes the ideal inference functionality; the simulator for a network is the composition of the per-gadget simulators. We therefore do not prove "private CNN inference" as a monolith: we exhibit a handful of gadgets and compose them. Swapping in more channels, more convolution blocks, or residual adds requires no new security argument, only a new sequence of gadget calls.

7 Security Model

Threat model. Semi-honest adversary; honest-majority three-party MPC for the nonlinear segment; the FHE evaluator is an untrusted server. Only the final output (the class index) is revealed. Intermediate activations remain shared or encrypted, and each switch opens only a uniformly-masked value, which is simulatable.

Disclosed implementation gaps. We distinguish the design from its current implementation.

Table 1: The gadget library as compositions of ABB primitives.

gadget	definition	primitives
`relu`	$x \cdot (0 < x)$	`sub`, `lt`, `mul`
`max2`	`mux`$(a < b,\, b,\, a)$	`lt`, `sub`, `mul`, `add`
`maxpool2d`	fold of `max2` over a window	(above)
`argmax`	scan tracking $(\text{best}, \text{idx})$	`lt`, `mul`, `add`, `share_public`
`dense` / `conv2d`	$\sum W \cdot x + b$	`mul`, `add`

1. **Threshold decryption is simulated.** The masked decryption uses a single client key as a stand-in for threshold FHE [3]. The data flow is already correct — only uniformly-masked values are opened — so a deployable instantiation secret-shares the FHE key without changing the protocol logic.

2. **The fast reference FHE engine is insecure; the secure one is RLWE.** We port scale-invariant NTRU [23, 5] as a speed reference only. It is overstretched even at depth one: the NTRU fatigue point [2, 19, 13] for the parameters a usable plaintext modulus forces is well below the required q, so we do not claim it secure. The secure instantiation is RLWE (BFV), which has no NTRU lattice and no fatigue point, and supports the identical depth-one, no-relinearization trick. *All headline accuracy and correctness numbers are measured on BFV.* A machine-checked regression asserts the ring dimension and modulus are within the standardized 128-bit budget [1].

3. **Malicious security is out of scope**, as is standard for this line of work.

8 Evaluation

We implement the full construction in a single Rust crate (no GPU). The three MPC parties run a real replicated-sharing wire protocol over serialized messages, with a network-cost model (round-trip time and bandwidth). Every benchmark verifies the encrypted result against a plaintext integer reference and reports `correct`; all numbers are emitted by the tool, not hand-entered. Wall-clock figures are single-thread on commodity hardware and are meant to be read as ratios, not absolutes.

8.1 Correctness and the zero-bootstrap invariant

At every measured depth and width, the encrypted class equals the plaintext class *bit for bit*, and the bootstrap counter is exactly zero. This is the operational content of Theorem 1: had noise accumulated with depth, deep outputs would diverge from the plaintext reference; they do not. The same invariant holds for the fully encrypted two-convolution CNN of Section 8.4, whose four encrypted linear layers and three switches all report zero bootstraps.

8.2 The MPC segment: making it private, then fast

Table 2 traces the MPC nonlinearity segment of the flagship CNN (28×28 input) through the sequence of protocol changes described in Section 5. An initial masked-open comparison was found to leak its input (the mask carried no secret entropy) and was replaced by the ABY3 bit decomposition; correctness held throughout, but the correct protocol costs more. Eliminating Beaver triples (PRF resharing) then cut bandwidth 15×, and batching the decomposition across each layer cut time a

Table 2: MPC nonlinearity segment of the 28×28 CNN across protocol revisions. Every row is bit-exact against the plaintext reference; only privacy and cost differ.

protocol stage	MPC time	wire bytes
leaky masked-open (insecure baseline)	17.6 s	1.2 MB
correct bit decomposition, Beaver triples	76.9 s	159.4 MB
+ triple-free PRF resharing	40.8 s	10.4 MB
+ batched ReLU decomposition	20.7 s	10.2 MB
+ batched max-pooling	**6.1** s	10.0 MB

Table 3: Full-test-set accuracy of the deployed integer model, by architecture. All fit the depth-one noise budget (nbits ≤ 30).

architecture	MNIST test accuracy
$\texttt{conv}(8) \to \texttt{dense}(10)$	94.96%
$\texttt{conv}(12) \to \texttt{dense}(10)$	95.73%
$\texttt{conv}(8) \to \texttt{dense}(128) \to \texttt{dense}(10)$	97.63%
$\texttt{conv}(8) \to \texttt{dense}(256) \to \texttt{dense}(128) \to \texttt{dense}(10)$	97.89%
$\texttt{conv}(8) \to \texttt{conv}(16) \to \texttt{dense}(128) \to \texttt{dense}(10)$	**98.05**%

further 6.7×. The endpoint is *private, triple-free, and batched* — and 2.9× faster than the original leaky baseline it started from.

8.3 Convolution packing

Replacing the im2col/Toeplitz convolution with the reversed-axis packing of Section 4 reduces the number of ciphertext multiplications from hundreds to one per output channel, independent of image size. On the 28×28 classifier the convolution's FHE time drops from 77.5 s to 0.26 s (294×), and the FHE share of runtime falls to a few percent — confirming that the MPC nonlinearities, not the FHE arithmetic, are the cost center that Section 5 then addresses.

8.4 Real MNIST: a fully encrypted two-convolution CNN

We train the exact architecture the pipeline evaluates — $\texttt{conv}(8, 3 \times 3) \to$ ReLU $\to$ pool $\to$ $\texttt{conv}(16, 3 \times 3) \to$ ReLU $\to$ pool $\to$ $\texttt{dense}(128) \to$ ReLU $\to$ $\texttt{dense}(10)$ — on the full 60,000-image MNIST [21] training set, then *post-train quantize into the exact integer fixed-point scheme the encrypted pipeline runs*. The reported accuracy is thus that of the deployed integer model, not of a float model that would lose accuracy under quantization. This model has four encrypted linear layers (two convolutions, two dense), three MPC nonlinearities, and three scheme switches, and the second convolution is multi-channel and packed with the same reversed-axis layout.

Because the construction makes depth free, we used it as an instrument to search architectures (Table 3): a second convolution layer beats both a wider single convolution and a deeper fully-connected stack — spatial depth is what MNIST rewards, and the construction supplies it at no cryptographic cost. The chosen two-convolution model attains 98.05% on the 10,000-image test set. The Rust integer forward pass agrees with an independent numpy reference digit-for-digit, and the *encrypted* pipeline reproduces the plaintext prediction on every real digit tested, at zero bootstraps. Because the hybrid is bit-exact, the encrypted accuracy *equals* the plaintext accuracy of 98.05%:

Table 4: Positioning on MNIST. The comparison axis is *what is hidden*, not accuracy.

system	MNIST acc	what is hidden
this work	**98.0**%	**image *and* weights** (ct×ct)
Gazelle [18]	∼99%	image only (plaintext weights)
Delphi [24]	∼99%	image only
CrypTFlow2 [26]	∼99%	image only
MiniONN [22]/Cheetah [16]	∼99%	image only

privacy costs nothing in accuracy, demonstrated on real inputs rather than asserted.

8.5 Positioning against prior inference systems

Table 4 places the result against representative two-party inference systems. The residual $\sim 1\%$ accuracy gap is model size, a training question the construction is indifferent to. The distinguishing axis is the last column: prior systems are client/server protocols in which the server evaluates on *plaintext* weights, so the model is not hidden; here every linear layer is ciphertext×ciphertext, so the weights are encrypted too, and only the class index leaves the cryptographic domain.

9 Related Work

Hybrid linear-FHE / nonlinear-MPC inference. Gazelle [18], Delphi [24], Cheetah [16], and CrypTFlow2 [26] pioneered evaluating linear layers under HE and nonlinearities under MPC. They treat the switch as engineering overhead and keep the model in plaintext. We identify the switch as a bootstrapping-elimination mechanism enabling unbounded depth with a depth-one FHE engine, and we encrypt the weights.

Preprocessing. Delphi [24] moves query-independent HE work off the critical path. The same idea applies to the scheme switch: mask and return-path randomness are query-independent and can be precomputed, so the online return leg becomes a plaintext addition.

Scheme switching between FHE schemes. CHIMERA [6] and PEGASUS [17] switch FHE↔FHE to pick the cheapest scheme per operation. We switch FHE↔MPC and exploit the re-encryption as a noise refresh.

Leveled and bootstrapped FHE. BGV [7], BFV [14], and CKKS [9] evaluate bounded-depth circuits and bootstrap (following Gentry [15]) to go deeper; TFHE [10] bootstraps every gate. Our construction removes the bootstrap for the deep-inference workload by interleaving an MPC round trip that already exists for the nonlinearity.

Overstretched NTRU. The insecurity of NTRU at large modulus is established by [2, 19] and sharpened by [13]; we rely on it to justify using NTRU only as a non-secure reference and RLWE as the secure instantiation.

10 Limitations

The MPC nonlinearity and the conversions dominate cost: the construction wins on depth scaling, on encrypting the model, and on the thin-pipe regime, *not* on a single small layer over a fast LAN, where pure MPC is faster. Softmax and layer normalization (as in transformers) need additional MPC subprotocols (exponentiation, reciprocal, inverse square root) that we do not implement; deep MLP/CNN ReLU stacks are supported. Layer width per ciphertext is bounded by the ring dimension through the packing constraint; wider layers chunk. Multi-channel inputs beyond the ring-dimension bound (e.g. CIFAR-scale three-channel images) require splitting channels across ciphertexts or a larger ring — a packing question, not a limit of the construction.

11 Conclusion

The scheme switch that hybrid private-inference systems already perform for each nonlinearity is, at the same time, a noise reset: it re-encrypts the activation to a fresh, depth-one ciphertext. Reading the switch this way turns bootstrapping from a per-depth cost into an absent one, and lets the FHE engine be a minimal, evaluation-key-free, single-multiply primitive. We gave the argument precisely, instantiated it with both operands encrypted, built a triple-free batched MPC layer, and validated the whole pipeline bit-exactly — culminating in a fully encrypted two-convolution CNN at 98.05% on real MNIST with zero FHE bootstraps.

References

[1] Martin Albrecht et al. Homomorphic encryption security standard. HomomorphicEncryption.org, Toronto, Canada, 2018.

[2] Martin R. Albrecht, Shi Bai, and Léo Ducas. A subfield lattice attack on overstretched NTRU assumptions. In *CRYPTO*, pages 153–178, 2016.

[3] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In *EUROCRYPT*, pages 483–501, 2012.

[4] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, pages 420–432, 1991.

[5] Joppe W. Bos, Kristin Lauter, Jake Loftus, and Michael Naehrig. Improved security for a ring-based fully homomorphic encryption scheme. In *IMA International Conference on Cryptography and Coding*, pages 45–64, 2013.

[6] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. CHIMERA: Combining ring-LWE-based fully homomorphic encryption schemes. volume 14, pages 316–338, 2020.

[7] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory*, 6(3), 2014.

[8] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 136–145, 2001.

[9] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, pages 409–437, 2017.

[10] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. volume 33, pages 34–91, 2020.

[11] Ivan Damgård and Jesper Buus Nielsen. Universally composable efficient multiparty computation from threshold homomorphic encryption. In *CRYPTO*, pages 247–264, 2003.

[12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*, pages 643–662, 2012.

[13] Léo Ducas and Wessel P. J. van Woerden. NTRU fatigue: How stretched is overstretched? In *ASIACRYPT*, pages 3–32, 2021.

[14] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. 2012. 2012/144.

[15] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *41st ACM Symposium on Theory of Computing (STOC)*, pages 169–178, 2009.

[16] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In *31st USENIX Security Symposium*, pages 809–826, 2022.

[17] Wen jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1057–1073, 2021.

[18] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium*, pages 1651–1669, 2018.

[19] Paul Kirchner and Pierre-Alain Fouque. Revisiting lattice attacks on overstretched NTRU parameters. In *EUROCRYPT*, pages 3–26, 2017.

[20] Peter M. Kogge and Harold S. Stone. A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Transactions on Computers*, C-22(8):786–793, 1973.

[21] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[22] Jian Liu, Mika Juuti, Yao Lu, and N. Asokan. Oblivious neural network predictions via MiniONN transformations. In *ACM CCS*, pages 619–631, 2017.

[23] Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *44th ACM Symposium on Theory of Computing (STOC)*, pages 1219–1234, 2012.

[24] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. DELPHI: A cryptographic inference service for neural networks. In *29th USENIX Security Symposium*, pages 2505–2522, 2020.

[25] Payman Mohassel and Peter Rindal. ABY3: A mixed protocol framework for machine learning. In *ACM CCS*, pages 35–52, 2018.

[26] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *ACM CCS*, pages 325–342, 2020.